Hexagonal Architecture (Ports and Adapters)

the core defines the plugs

Watch the 4-min video

> **In one line:** the domain core defines **ports** (interfaces in its own terms); **adapters** implement them for real technologies or call into the core; all source dependencies point inward, so infrastructure can be swapped and the core tested with fakes.

1 Tangled logic

A pricing rule reads HTTP headers, writes SQL inline and calls the OpenAI SDK directly. Now you must switch LLM provider, or simply write a unit test, and everything is tangled. **Hexagonal architecture** (Alistair Cockburn, 2005), also called **ports and adapters**, keeps the core separate from the outside world.

2 Core, ports and adapters

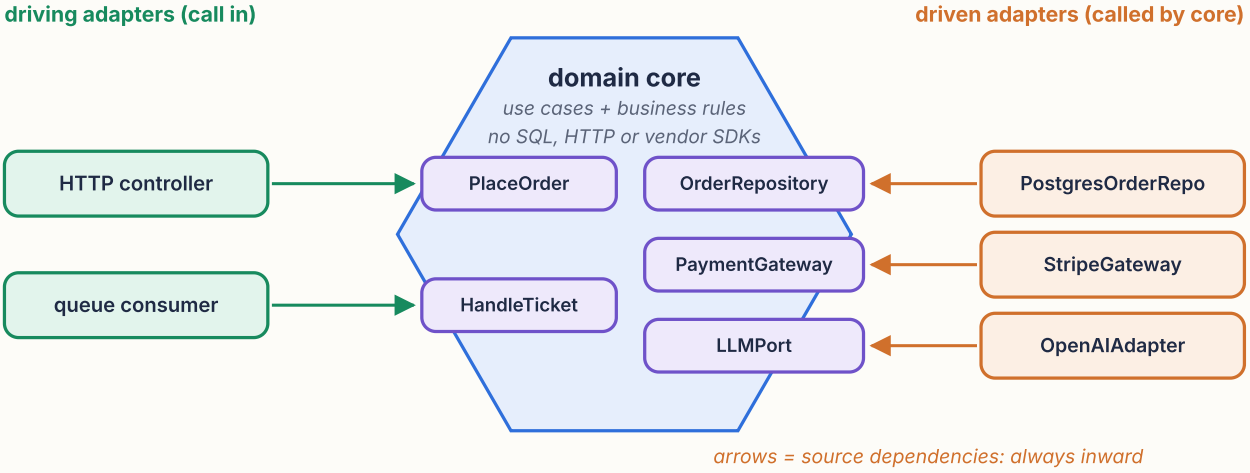


Figure 1. The core exposes and defines ports; adapters on both sides plug in. The six sides have no meaning: they just leave room for many ports.

- **Domain core:** business rules and use cases. Knows nothing about databases, web frameworks or vendors.
- **Ports:** interfaces the core defines in its own language: `OrderRepository.save(order)`, `PaymentGateway.charge(amount)`. Never "execute SQL" or "POST JSON".
- **Driving (primary) adapters** call into the core: an HTTP controller, a CLI command, a queue consumer, a test.
- **Driven (secondary) adapters** are called by the core: a Postgres repository, an email sender, a Kafka publisher, an LLM client.

3 Dependency inversion

The core defines the interface and the adapter implements it, so the **Postgres adapter depends on the core**, never the other way round. That is the D in SOLID (dependency inversion principle) applied to the

whole application. Control flows outward at run time (core → port → adapter → database), but source dependencies point inward. A **composition root** (the app's start-up code or a DI container) chooses which adapter plugs into each port.

4 Testing



Figure 2. Same core, different wiring. Only the composition root changes.

Plug in-memory fakes into the ports and the business logic is tested in milliseconds, with no database, network or API keys. Then test each real adapter with a few **integration tests** against the real technology (e.g. Postgres in a container). Fast tests where the logic is, slow tests only at the edges.

Use it when	Skip it when
the domain logic is rich	simple CRUD: forms in, rows out
you integrate several external systems	a prototype or short-lived script
you expect to swap infrastructure or vendors	the framework will never change and there is little logic

Don't add a port for everything. A good rule: one port per external system, or per reason to change.

WORKED EXAMPLE: SWAPPING THE LLM PROVIDER

A support assistant's core defines `LLMPort` with one method, `complete(prompt: str) -> str`. An `OpenAIAdapter` implements it. Later the company moves to **Azure OpenAI** for data residency. The team writes `AzureOpenAIAdapter(LLMPort)` and changes one line in the composition root. No business rule changes. Unit tests use a `FakeLLM` that returns canned answers, so they run offline and are deterministic.

COMMON MISTAKES

- Ports that mirror a vendor's API (`OpenAIChatCompletionPort`) instead of the core's need.
- Domain objects importing ORM base classes or framework annotations.
- An interface for every class, "just in case".
- Only fake-adapter tests, with no integration tests of the real adapters.

5 Key terms

Port: interface defined by the core in its own terms

Adapter: implements a port for a real technology

Driving adapter: calls into the core: HTTP, CLI, consumer

Driven adapter: called by the core: DB, email, LLM

Dependency inversion: details depend on abstractions, not the reverse

Composition root: start-up code that wires adapters to ports

6 Interview questions

Q1. Explain ports and adapters. BASIC
The core defines ports, interfaces in its own terms; adapters implement them for real technologies or drive the core; dependencies always point inward so infrastructure can be swapped.

Q2. What's the difference between driving and driven adapters? BASIC
Driving adapters call into the core (HTTP controller, queue consumer); driven adapters are called by the core through a port (database, email, payment API).

Q3. How does it help testing? SCENARIO
You plug in-memory fakes into the ports, so business logic is tested in milliseconds with no database, network or API keys; real adapters get a few integration tests.

Q4. You must move from OpenAI to Azure OpenAI. What changes? SCENARIO
Write a new adapter implementing the same LLM port and change the wiring in the composition root; the core and its tests don't change.

Q5. If the core calls the database at run time, how do dependencies point inward? TRICKY
The core calls an interface it owns; the database adapter implements that interface, so source code depends inward while control flows outward.

Q6. When is hexagonal architecture overkill? TRICKY
For simple CRUD services or prototypes with little logic, where many interfaces add ceremony without protecting anything.

Next: SD-L-014 · Clean Architecture backendroadmap.com